

程序设计的实验部分

Bring color to knowledge

Author: 胡潇天 (后简称活性炭)

Institute: 假装有工作室

Date: 2025 年 12 月 26 日

Version: v6.0.0

邮箱: 1985716106@qq.com



人生是一场漫长的旅程，不在乎目的地，而在乎沿途的风景以及看风景的心情。让我们带着希望和勇气，去探索未知的世界，去追求更高的目标



第 6 章 lab6 2048

❤无畏孤单，因为这个世界上，肯定有一个人正努力的走向你.❤

——《龙猫》

6.1 介绍

2048 是一款风靡全球的游戏，而且诞生了很多别的同类型合成游戏, 他的唯一目的就是通过上下左右键操作，来最终合成 2048

6.1.1 项目

下载 lab6.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug
- CMakeLists.txt
- Lab6.cpp

6.1.2 学习内容

lab6 的实验可能没有前面那样想让你刻意的去使用 STL 中的很多容器，他的目的反而是让你去体会一下面向对象的小型开发，更多的考验你的思维，而且不用记忆那么多复杂的函数

- 学会对数组进行合并
- 学会移动数组

6.1.3 完成内容

对于 2048 而言其实，就三个操作，按 WASD 来移动砖块，然后合并砖块，判断游戏结束。

6.1.4 前置内容

一点变量名字的使用:

- board[4][4] 整个游戏的 4*4 砖块
- score 目前的得分

开始前说明：请不要过多的关注于整个游戏是如何实现的，而是只聚焦你要实现的部分。

6.2 Getting Started

6.2.1 Phase 1

实现 bool compressAndMerge(), 该函数有一个参数

- row[4]: 从期盼中取出的一行/一列四个元素。

这个函数的作用是合并砖块，然后返回是否发生了合并。合并可以一步一步来进行

6.2.1.1 横向移动

1. 将所有非 0 数字按原顺序移到最左边

效果：$[2,0,0,2] \longrightarrow [2,2,0,0]$

Axiom 6.1 (Hint)

- *需要开一个长度为 4 的数组，然后将所有非 0 元素暂存起来*

6.2.1.2 压缩合成

2. 将相同元素合并起来效果：$[2,2,4,4] \longrightarrow [4,0,8,0]$

Axiom 6.2 (Hint)

- *增加得分（2048 规则：得分等于新生成的方块值）*
- *将后位清零，防止在同一次移动中发生链式合并*

6.2.1.3 再次压缩

3. 移除步骤 2 合并过程中产生的“0”间隙

效果：$[4,0,8,0] \longrightarrow [4,8,0,0]$

6.2.1.4 搬运

4. 我们整个过程如果运用了临时数组来处理，那么还需要搬运一步来完成从临时数组到 row 数组的搬运，如果只是在原数组上进行操作，那么可以不用这一步。

友情提示

如果只是将代码塞给 AI 的话是体会不到整个设计游戏的快乐过程的，助教认为初上手游戏制作快的可能需要三四个小时，慢的可能需要七八个小时来完成。听从自己的本心去写代码，不论结局如果，自己写的一定是最具有价值的。

6.2.2 Phase 2

实现 move() 函数，该函数有两个参数

- direction: 移动的方向, 传入 WASD 四个方向
- addRandom: 无需在意该参数，只是为了测评用

该函数就是需要实现上下左右移动了, 实现流程大致如下

1. 将传入的方向键小写
2. 判断四个方向
3. 然后遍历每行/每列，依次通过之前 Phase 1 的合并规则，合并每行/每列
4. 修改整个 board 图。

为了降低大家的难度，这里提供一个方向的参考代码

```
if (direction == 'd') {
        for (int i = 0; i < 4; ++i) {
            int row[4] = {board[i][3], board[i][2], board[i][1], board[i][0]};
            if (compressAndMerge(row)) {
                changed = true;
                board[i][0] = row[3]; board[i][1] = row[2];
                board[i][2] = row[1]; board[i][3] = row[0];
            }
        }
}
```

由于我们的设计仍存在发展阶段，这里可以提供一种思路就是将上下右四个方向映射成向左的方向，只是看的方向不一样，但是逻辑是相同的, 当然你写成四个 if 语句也没有问题。

如果能这样写出来代码 (前提是自己写的) 可以提供一部分小小的加分，鼓励大家的创新思维。

6.2.3 Phase 3

实现 checkGameOver() 函数，该函数无参数

这个函数用来判断游戏是否结束，这个函数的情况有很多，场上还存在空白方块（值为 0），存在可合成方块（相邻相同），此部分就是开放式的发挥了，比如有人就不想玩到 2048 结束，当然可以玩到 4096。自行发挥即可

友情提示

接下来就是玩的阶段了，就请主函数中将模式切换到 play(), 提交代码的时候切换到 test(), 由于代码的一些局限性，测试部分主要测试大家的合成和移动规则，结束规则可以自行拟定。

最终效果如下图

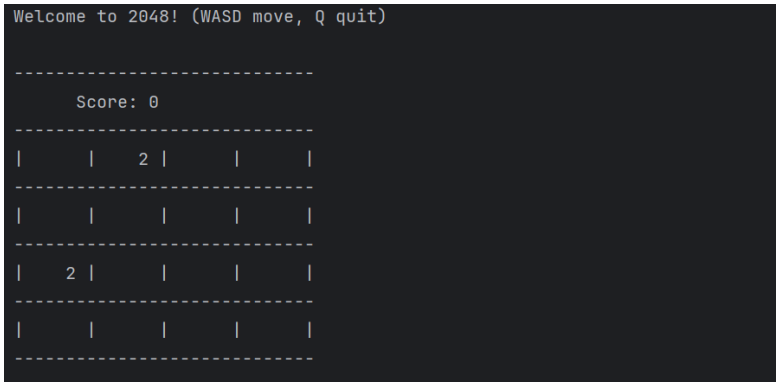


图 6.1: 游戏运行图

6.3 submit

6.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，提交到 cpp 后缀的文件，同时将模式切换到 test().

6.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

6.4 References

- 由于很多版本是 java 和 python 的，C++ 写的很少，而且原理差距很大，建议参考文档